WERGONIC · ENGINEERING · EPIC

Backend automated test suite

Calculation logic & tenant isolation

Repo: wergonic-django-backend · Branch: dev · 5 August 2026

Why this exists

The backend has about 17 real test methods across 14 apps — most `tests.py` files are empty stubs — and no test infrastructure. CI runs `manage.py test`, finds almost nothing and reports green. So "green" today means **nothing is tested**, not "nothing is broken".

Meanwhile two audits in July found real calculation errors and tenant-isolation defects, where one company's data is reachable from another company's account. This epic builds a real suite, starting from the code we already know is fragile.

Goal

- Stand up backend test infrastructure — there is none today.
- Lock down the calculation and permission defects with tests, so the fixed ones can't come back and the open ones can't ship half-fixed.
- Get a baseline suite that CI actually gates on, at the level the Flutter repo already runs at.

Definition of done — per sub-task

- The module has tests for its main path plus the specific edge and boundary cases listed.
- Where a note says the code is wrong today, the test **fails first and passes once it's fixed**. Seeing it go red is the proof the test is real.
- The suite runs green in CI, wired into the GitHub Actions workflow.
- No test touches the staging or production DB, Firebase, or live data. Tests run on isolated test data only.

How to read this

Each row is one target: the file and function on the left, what the test has to prove on the right. The right column is the instruction, not a description of the module — those are the things that must be asserted. Whatever else the module needs is your call, and anything you find beyond the list is worth adding.

A few notes say the current behaviour is wrong. Those are deliberate: the code reads as intentional in those places — one of them even has a comment defending the wrong value — so don't take today's behaviour as the expected result there. Everything below was checked against `dev` on 5 August 2026.

The phases are priority order. Phase 1 and 2 are the must-ship, together they cover every defect we know about. Phase 3 and 4 are backlog.

READ THIS BEFORE WRITING EXPECTED VALUES

Four rules that decide whether your assertions are right

- **Head backward bending is −5°, trunk backward bending is −10°.** They are different on purpose — never assume they share a threshold. Assert them side by side so nobody "harmonises" them later.
- **The head value still has to be changed in one place:** the risk verdict in `risk_assessment_posture` and the Word report (item 2.18), which both still use −10°. `ramp_calculator` already uses −5° correctly — it only has stale `gt_10` key names to rename.
- **`calculate_postures_2` returning a 1.5 score is wrong** and is still in the code (item 1.2). Reproduce it red first.
- **The overlapping wording in `posture_thresholds` is intentional** — test the tables as pure logic, don't "fix" them.

Sub-task 0 · Test infrastructure

prerequisite

Nothing below can run without this. There is only a bare test runner today and no reusable fixtures. The approach and tooling are your call.

- A working test-run setup that CI executes and gates on — today's CI step is near-empty and always green.
- Reusable fixtures/factories for the core models the tests need: `User`, `Organization`, `UserOrganization`, `Session`, `Measurement`, `MeasurementData`, `Event`, `Category`, `Label`, `SessionActivity`, `SessionCondition`, `WorkCycleEvent`, `RAMPAssessment`, `SessionStats`, `Ticket`, `Notification`.

Phase 1 · Pure calculation logic

do first

Best value per hour: pure functions, called with dicts, lists and DataFrames, no DB. Kills the worst calculation findings fast.

#	MODULE	WHAT THE TEST MUST PROVE
1.1	mec_mapping.py `calculate_time`	Generic band accumulator, threshold-agnostic — no −5/−10 baked in, callers pass the threshold. Upper-inclusive (`v ≤ upper`), lower-exclusive (`v > lower`): for band `(20,45]`, 20 is out, 45 is in, 45.1 is out. `side_bending` uses `abs()`; `elevation_angle` drops `v < 0`; percentages are time-weighted (`matching_ms / valid_ms`); empty input → `{0,0,0}`.
1.2	mec_mapping.py `calculate_postures_1..6`	**Wrong today:** `calculate_postures_2` returns 1.5 for the input band `[1, 6.25)`. A RAMP score is never 1.5. Before the fix: reproduce the 1.5 (red). After: it never returns 1.5 and scores stay monotonic in %. Don't hard-code the new band values until the fix is defined. Also: `None → None`; clamp `<0 → 0` and `>100 → top`; ladders 1, 3, 4, 5, 6 never emit 1.5.
1.3	mec_segments.py `scale_results_with_takt_time`, `get_feedback_frequency`	`takt=60, count=10, static=120` → `600 / 120`. `takt ≤ 0` raises. Frequency: metadata `None` → 52 for hand, 13 otherwise; arm variants fall through to the standard branch.
1.4	posture_thresholds.py `3 functions`	Pure threshold tables that feed every report. The overlapping wording in them is intentional — test as-is, don't change it. Each boundary lands in exactly one band; ranges end at `(100, 'high')`; integers render without a trailing `.0`.

Phase 2 · Tenant isolation & integration

The cross-org data-leak work. Test the shared gate *and* its callers — the scoping fixes all delegate to a couple of shared functions, which are the real single points of failure.

#	MODULE	WHAT THE TEST MUST PROVE
2.1	users/scoping.py `get_request_organization`	**Start here.** About 15 endpoints delegate to this one function. `X-Client-Platform: web` → consulting org vs active org; `_cached_org` memoization; unauthenticated short-circuit. If this returns the wrong org, every scoped view silently breaks while its own test still passes.
2.2	user_sessions/permissions.py `3 classes`	The gate other views assume is "already guarded", never tested. **Wrong today:** `SessionCrudPermission` calls `session.worker.user`, which raises `AttributeError` when `worker` is `None` — same root cause as 2.12. Give that its own case.
2.3	work_tags/permissions.py `4 classes`	Org-admin and ergonomist gates; cross-org → `False`; `is_deactivated=True` → `False`; swagger bypass behaviour. The classes themselves are fine — it's the views that don't use them, see 2.5.
2.4	users/permissions/ `level_0/1/2 + userCrud`	The role-based CRUD permission matrix.
2.5	work_tags/views.py `Event / Category / Label`	**Wrong today:** the org-scoping `permission_classes` are commented out and replaced with plain `IsAuthenticated`, so any logged-in user reads and writes any org's events, categories and labels. Test through `APIClient`: `org_id` and `category_id` from the URL must never be trusted over the caller's own org.
2.6	tickets/views.py	**Wrong today:** `TicketRetrieveDestroyView` is `Ticket.objects.all()` with only `IsAuthenticated`, so any ticket id can be read or deleted by anyone. Test that tickets are scoped to the org.
2.7	stats_view.py `GenerateGeminiAnalysisView.get`	Already correct — the view compares `session.organization_id` against the active org. The test keeps it that way: the AI report GET rejects another org's session.
2.8	work_library/views.py `+ serializers.py`	Already correct — querysets are scoped and a cross-org group raises a validation error. The test keeps it that way: an update can't re-point a record to another org's activity.
2.9	workcycles/views.py `TaskViewSet.get_queryset`	Already correct — the queryset filters by the request org. The test keeps it that way, including that org-less orphan tasks are not exposed across companies.
2.10	ramp_calculator.py `run_ramp_calculations`	Head backward already uses `upper_threshold = -5` — assert −5, not −10. The response keys are still named `postures_2_…gt_10_degrees_*`; after the rename they read `gt_5`. Trunk posture 4 uses −5 and doesn't change. This function must emit **no −10 value at all** — the −10 trunk verdict lives in 2.18, keep the two from getting mixed up. Task-scoped posture 6 uses the task window, not the whole session.
2.11	mec_segments.py `compute_mec_for_segment`	DB-backed. Trunk with side-bending present but forward-bending null → no swallowed `UnboundLocalError`, and the missing-limb flags get set. Worth a look while you're in there: the head branch checks whether `forward_bending` data exists, then collects `side_bending` values.

#	MODULE	WHAT THE TEST MUST PROVE
2.12	management/commands/ `session_check.py`	**Wrong today:** it dereferences `session.worker.user.organization`, so the per-minute auto-finish crashes on a session whose worker was deleted. Test that it survives one.
2.13	management/commands/ `archive_sessions.py`	The status to match comes from the rule row in the DB, not from code — so this is about the stored rule values, not the command. Test that archiving works with the current status values, after the ONGOING/FINISHED/INTERRUPTED rename.
2.14	notifications/ `models.py + views.py`	**Wrong today:** the notification is stamped with the recipient's active consulting org, not the org the session belongs to. Test that a failed-measurement notification is filed under the session's org.
2.15	stats_view.py `single-session Excel export`	**Wrong today:** head is written into the same columns as trunk (D/F for forward, J/L for side) right after trunk, so head overwrites it. Test that head values don't land in the trunk columns.
2.16	measurements.py `merge_measurment_files`	Already correct. The test keeps it that way: the merged zip includes `_metadata_merged.json`, the highest `App.Version` is chosen, the axis swap is correct.
2.17	session_views.py `session merge`	Already correct — measurements are deliberately not moved, the async task rebuilds them from the combined file. The test locks that in end to end, which is why this one is heavier than the rest: it needs the full merge → reprocess pipeline.
2.18	risk_assessment_posture.py `calculate_risk_posture_stats` + session_docx_report.py	**Wrong today, and it looks intentional in the code — there's a comment defending −10°.** This is where the head threshold actually changes. The head backward risk verdict goes to −5° (mask ≤ −5: a head row at −6° starts counting; the id and label say −5°). The **trunk verdict stays at −10°** — trunk −7° doesn't count, −10° and −12° do. Assert head and trunk side by side. In the Word report the head row is labelled −5° and the trunk row −10°.

Phase 3 · Important logic

backlog

Not covered by the audits, medium risk: filters, report maths, signal-processing helpers.

AREA	MODULES
Signal processing	`mec_mapping.py` — peak detection, static periods, butter filter, arm postures, zones, hourly
Wrist analysis	`wrist_mec_mapping.py` — position, custom zones, peaks, static
Report maths	`report_service.py` — label-bounds parsing, bin midpoints, weighted mean/median, option and velocity blocks
Distance zones	`transformations.py :: compute_distance_zone_minutes` plus the round/timezone helpers — the pure banding that feeds posture 6
Pie & risk stats	`pie_stats.py`, `risk_assessment_posture.py` — the banding. Guard the intentional constants; the head −5° change itself is in 2.18
Data reshaping	`stats.py`, pure helpers in `measurements.py`, `task_measurments.py`, `filters.py :: pair_measurements_by_time`, preprocessing and swapping

AREA	MODULES
DOCX helpers	`session_docx_report.py :: get_risk_image`, `format_duration_seconds`

Phase 4 · Nice to have

backlog

Lower risk or harder to test. The TNO pipeline, including `extractTaskCharacteristics`, sits here — it's deliberately parked for now. Also: inversion correction, time bounds, percentage change, swapped pie stats, user-service claims, report JSON and Excel value maps, the work_tags event upsert, temperature and percentile formatting in the reports, and `firebase_auth/authentication.py :: authenticate` — the token-parsing branch, worth a look because it silently returns `None` on an unknown uid.

Out of scope

don't spend time here

- Raw-SQL percentile paths in `stats.py` — Postgres/Timescale-only and would need huge fixtures. The reusable zone and pivot logic is covered elsewhere.
- Firebase and DB IO orchestration in `measurements.py`, `task_measurments.py`, `calculation_service.py` — the underlying maths is covered by the pure targets.
- DOCX/XLSX byte assembly (`build_session_word_report`, the Excel builders, `docx_pie_charts.py`) — the data-selection branching is covered through the helpers.
- Firebase Storage pass-throughs (`tickets/utils.py`, `devices/utils.py`) and user-service create/update/delete — heavy multi-system mocking for little return.